

Day 5: Lab Exercises — GPU Programming with Python

Duration: 4 hours (self-paced hands-on practice)

Reference: [day5/README.md](#), [day5/README-session1.md](#), and [day5/README-session2.md](#)

Note: These exercises are for personal practice to reinforce the concepts covered today. Start from Exercise 1 and work sequentially.

Exercise 1: PyCUDA Vector Scaling & Memory Management

Topics: PyCUDA, `cuda.mem_alloc()`, `memcpy_htod()`, `memcpy_dtoh()`, `SourceModule`

Problem Statement

Write a script `ex01_pycuda_vector_scale.py` that scales a 1D vector by a scalar constant α on the GPU ($y[i] = \alpha \cdot x[i]$):

1. Host Preparation:

- Create an input vector x containing 65,536 elements of type `float32`.
- Allocate an empty output vector y of the same shape and type.
- Choose a scaling factor $\alpha = 3.5$.

2. CUDA Kernel Formulation:

- Write a CUDA C kernel inside a Python string:

```
__global__ void scale_vector(float *y, const float *x, float
alpha, int n) {
    int idx = blockDim.x * blockIdx.x + threadIdx.x;
    if (idx < n) {
        y[idx] = alpha * x[idx];
    }
}
```

3. GPU Memory & Execution:

- Allocate device memory on the GPU using `cuda.mem_alloc()`.
- Copy the input vector x from Host to Device with `cuda.memcpy_htod()`.
- Launch the kernel using a block size of 256 threads (`block=(256, 1, 1)`). Calculate the required grid size to cover all 65,536 elements.
- Copy the computed array y back to Host RAM with `cuda.memcpy_dtoh()`.
- Verify that all elements satisfy $y[i] = 3.5 \cdot x[i]$.

Hints & Guidelines

- If executing on a computer without an NVIDIA GPU, implement the architectural simulation fallback demonstrating the exact same 5-step data flow as seen in Day 5 Demo 01.
- Ensure all scalar parameters passed to the kernel (like `alpha` and `n`) are wrapped in NumPy fixed types (`np.float32(3.5)`, `np.int32(65536)`).

Best Practices

- Always check that your memory allocations and transfers match the exact byte size (`array.nbytes`).
- Minimize PCIe transfers: keeping data on the GPU across multiple kernel calls is key to achieving real speedups.

Exercise 2: 2D Matrix Transposition Kernel

Topics: 2D thread blocks, 2D grids, `threadIdx.x/y`, `blockIdx.x/y`

Problem Statement

Write a script `ex02_pycuda_matrix_transpose.py` that transposes an $N \times N$ matrix in parallel on the GPU:

1. Host Setup:

- Create a square matrix of size 512×512 with sequential values.
- Compute its expected transpose locally on the CPU for verification.

2. 2D CUDA Kernel:

- Write a CUDA kernel that maps 2D block and thread coordinates:

```
__global__ void transpose_matrix(float *out, const float *in, int
width, int height) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (col < width && row < height) {
        int in_idx = row * width + col;
        int out_idx = col * height + row;
        out[out_idx] = in[in_idx];
    }
}
```

3. Grid & Block Dimensions:

- Define a 2D block size: `block=(16, 16, 1)` (giving $16 \times 16 = 256$ threads per block).
- Define a 2D grid size: `grid=(width // 16, height // 16)`.
- Launch the kernel and copy back the transposed matrix.
- Assert that the GPU output matches the CPU transpose.

Hints & Guidelines

- In CUDA, the X dimension typically represents the column index, and the Y dimension represents the row index.
- Flattened index formula in row-major order: $\text{index} = \text{row} * \text{width} + \text{col}$.

Best Practices

- Use 2D thread blocks of 16×16 or 32×32 for image processing and matrix calculations.
- Always check array boundary conditions (`if (col < width && row < height)`) to prevent out-of-bounds VRAM access.

Exercise 3: High-Level GPU Arrays & Custom Elementwise Expressions

Topics: `pycuda.gpudarray`, `ElementwiseKernel`, GPU vector math

Problem Statement

Write a script `ex03_gpudarray_expressions.py` demonstrating high-level PyCUDA abstractions:

1. GPUDArray Vector Arithmetic:

- Create two vectors u and v with 500,000 random values.
- Transfer them to the GPU using `gpudarray.to_gpu()`.
- Compute the linear combination $w = 2.0 \cdot u + 5.0 \cdot v - 1.0$ directly in GPU memory using standard mathematical operators.
- Fetch the result back to CPU memory using `.get()` and verify against CPU NumPy.

2. Custom ElementwiseKernel (ReLU Activation Function):

- In deep learning, the Rectified Linear Unit (ReLU) is defined as $f(x) = \max(0, x)$.
- Create an `ElementwiseKernel` that applies ReLU to an array:

```
relu_kernel = ElementwiseKernel(
    "float *out, const float *in",
    "out[i] = (in[i] > 0.0f) ? in[i] : 0.0f",
    "relu_activation"
)
```

- Pass an array with both positive and negative values through the GPU kernel.
- Verify that all negative values are clamped to zero.

Hints & Guidelines

- `ElementwiseKernel` automatically handles block/grid dimensions, memory indexing, and compilation behind the scenes.
- `gpudarray.GPUDArray` provides convenient methods like `.sum()`, `.max()`, and `.min()` that execute as parallel GPU reductions.

Best Practices

- Prefer `gpuarray` and `ElementwiseKernel` over raw `SourceModule` C strings whenever performing element-wise array operations.
- Avoid transferring intermediate arrays back to the CPU if they will be needed by subsequent GPU operations.

Exercise 4: Pure Python GPU Kernels with Numba (`@cuda.jit`) & Shared Memory

Topics: Numba CUDA, `@cuda.jit`, `cuda.grid()`, `cuda.shared.array()`, `cuda.syncthreads()`

Problem Statement

Write a script `ex04_numba_shared_memory.py` that implements a 1D moving window filter using Numba CUDA:

1. The Goal:

- Compute a 3-point moving average for a 1D signal: $y[i] = \frac{x[i-1] + x[i] + x[i+1]}{3}$.

2. Writing the Numba Kernel:

- Write a kernel decorated with `@cuda.jit`:

```
from numba import cuda

@cuda.jit
def moving_average_kernel(input_data, output_data):
    # Allocate on-chip shared memory for the thread block:
    shared_tile = cuda.shared.array(shape=(256,),
                                     dtype=np.float32)

    t_id = cuda.threadIdx.x
    g_id = cuda.grid(1)

    # Load data from slow global VRAM into fast on-chip shared
    cache:
    if g_id < input_data.size:
        shared_tile[t_id] = input_data[g_id]

    # Synchronize all threads in the block to ensure loading is
    complete:
    cuda.syncthreads()

    # Compute moving average using fast shared cache:
    if 0 < t_id < 255 and g_id < input_data.size - 1:
        output_data[g_id] = (shared_tile[t_id - 1] +
                             shared_tile[t_id] + shared_tile[t_id + 1]) / 3.0
```

3. Launch the kernel using Numba execution configuration `[blocks_per_grid, threads_per_block]` and verify the filtered result.

Hints & Guidelines

- `cuda.syncthreads()` acts as a block-level barrier: no thread proceeds past it until all threads in that block have reached it.
- Shared memory lives in on-chip SRAM, offering up to $100\times$ lower latency than global VRAM.

Best Practices

- Use shared memory whenever multiple threads in a block need to read overlapping neighbors (stencils, convolutions, matrix multiplication tiles).
- Remember that shared memory is local to each thread block—threads in different blocks cannot access each other's shared memory.

Exercise 5: Cross-Platform Accelerator Discovery & OpenCL Kernels

Topics: PyOpenCL, platform discovery, device capabilities, OpenCL C kernels

Problem Statement

Write a script `ex05_pyopencl_runner.py` that explores cross-platform GPU programming using PyOpenCL:

1. Hardware Discovery:

- Query all available OpenCL platforms installed on the system using `cl.get_platforms()`.
- For each platform, iterate through its devices (`plat.get_devices()`).
- Print the device name, vendor, device type (GPU vs CPU), and maximum compute units.

2. Building and Running an OpenCL Kernel:

- Create an OpenCL context and command queue.
- Write an OpenCL C kernel that computes element-wise vector multiplication:

```
__kernel void vector_mult(__global const float *a,
                          __global const float *b,
                          __global float *c) {
    int gid = get_global_id(0);
    c[gid] = a[gid] * b[gid];
}
```

- Compile the program with `cl.Program(ctx, kernel_code).build()`.
- Create read-only buffers for inputs *a* and *b*, and a write-only buffer for output *c*.
- Enqueue the kernel across a global work-item size equal to the vector length.
- Read the result back into Host memory and verify correctness.

Hints & Guidelines

- In OpenCL, a single work-item corresponds to a CUDA thread, and `get_global_id(0)` corresponds to `cuda.grid(1)`.
- OpenCL code is vendor-neutral and will execute on NVIDIA, AMD, Intel, or Apple Silicon chips without code changes.

Best Practices

- Always check device properties (`device.max_work_group_size`) to ensure work-group sizes do not exceed hardware limits.
- Release or unmap buffers when no longer needed in long-running applications.